komplet SYSTEMBLUEPRINT & HANDLINGSPLAN – ALPHA Interface GUI (Blackbox E.Y.E), skal versioneres som v1.0, i både Markdown og PDF. inkluderer:

- Arkitekturdiagrammer (Mermaid)
- Modulbeskrivelser, CI/CD-struktur, sikkerhed
- Handlingsplan og roadmap
- Integrationsstrategi (BCP, Vault, etc.)
- Ansvars- og roller, revisionsmetadata og changelog

Blueprints uploades til:
/docs/SYSTEM_BLUEPRINT_AIG.md
/docs/SYSTEM_BLUEPRINT_AIG_v1.0.pdf

SYSTEMBLUEPRINT & HANDLINGSPLAN – ALPHA Interface GUI

🎯 **Formål**
Systemblueprintet beskriver ALPHA Interface GUI (AIG) som modulær webfrontend til BLACKBOX CODES-platformen (Blackbox E.Y.E). AIG's formål er at tilbyde et brugervenligt kontrolpanel for missions, intelligence, loghåndtering og værktøjer til agenter og administratorer. Systemet er *hybrid* (offline-first), med en lokal krypteret database (MySQL/MariaDB) og en cloud-backend (GreyEYE-platform) som fallback for synkronisering og realtidsdata.

Platformen er PHP/HTML/CSS/JS-baseret og designet til enterprise operations (OSINT, AI-assisteret missionstyring mv.). Alle API-kald er sikre (REST/JSON over HTTPS med OAuth2/JWT) og der er rollebaseret adgang (Admin vs. Agent) med PIN/Password-login (forberedt til 2FA/token).

🏗️ **Overordnet arkitektur**
AIG's arkitektur er modulær: Frontenden består af separate sider og komponenter for *Login*, *Dashboard*, *Missions*, *Intelligence*, *Logs* og *Tools* (jf. "modulbaseret navigation"). Backenden tilbyder REST-API'er (JSON/JWT) og WebSocket-forbindelser til realtidsdata og alarmer. Databasen indeholder modeller for agenter, missioner, logs mv., og scriptede migrations håndterer opgraderinger.

Integration med eksterne systemer (fx Hak5 Cloud C2, Blackbox Control Panel) foregår via dedikerede integrationsmoduler (/api/integrations/...) eller fælles eventbus/WebSocket-kanaler. Systemet er designet efter Blackbox-arkitekturstandarder (sikre forbindelser, audit logs, versionsstyring).

- **Moduler:** Login, Dashboard, Missions, Intelligence, Logs, Tools (jf. README). Hvert modul har egen frontend og API-endpoints.
- **Backend/API:** PHP-baseret REST-API med foruddefinerede ruter, inputvalidering og prepared statements. Alle kald autentificeres via JWT.
- **Dataflow:** Eksempelvis oprettes en mission via POST /api/missions → API → DB → notifikation til Dashboard/WebSocket for andre agenter. Intelligence-data hentes løbende fra eksterne OSINT-kilder. Logs indsamles fra UI- og API-hændelser til central audit.
- **CI/CD & Vault:** Systemet er "klar til" en CI/CD-pipeline og Vault/Secrets-håndtering. Deploy sker via automatiserede workflows (GitHub Actions/Jenkins) med staging/production-miljøer og rollback. Alle konfigurationer (DB-credentials, API-keys) skal hentes fra Vault eller Azure Key Vault.

🔧 **Komponentbeskrivelser**

Beskrivelse af nøglekomponenter:

- **Login/Session:** PHP-login-side (index.php) med PIN/Password. Anvender password_hash()/password_verify(), secure cookies og session-styring. 2FA (token) skal understøttes. Role-based access sikrer, at *Admins* kan se alle data, mens *Agenter* har begrænset udsyn.

- **Dashboard:** Oversigtsside med live-status (WebSocket-data), systemalarmer og hurtige genveje. Widgets kan udvides til at vise data fra andre moduler (f.eks. BCP-status eller missionstik). UI-komponenterne genbruges via PHP-includes og indlejrede JS-komponenter.
- **Missions-modul:** (UI & API) Flow for oprettelse, søgning, tildeling og opdatering af missioner. API-endpoints som GET/POST /api/missions, PUT /api/missions/{id} osv. Frontend-sider med oversigt, formular og historik. Tildeling sker med roles (Admin kan tildele til agent). Alle mission-hændelser logges.
- **Intelligence-modul:** Dashboard/rapporter for indsamlet OSINT-data og analyse. Henter eksternt feeds (f.eks. fra GreyEYE AI), viser kort og grafer. Skal kunne vise større oversigter, open report links. Backend-kald til eksterne API'er, webhooks (fx fra BCP, ID-Matrix) og intern søgning.
- **Logs-modul:** Centraliseret logvisning for alle systemhændelser (login, mission-ændringer, tools-kørsel). Logdata gemmes krypteret i DB. Eksport/mulighed for slet/excel. Loggen understøtter også audit-trail for eksterne systemer (BCP, GreyEYE) ved at modtage webhook-events.
- **Tools-modul:** Værktøjsbibliotek til eksekvering af "stealth tools" via AIG eller eksterne enheder. UI til at vælge redskab, udfylde parametre og starte job. Backend-endpoint som POST /api/tools/{tool}/execute. Resultater gemmes i logs og vises på dashboard. Kan integreres med BCP for decentrale executes.
- **Vault/Secrets-integration:** All konfigurationshemmeligheder (DB-credentials, API-keys) skal gemmes i Vault i stedet for i kode. Implementér PHP-klasse til at hente secrets fra Vault/Azure (f.eks. før db.php laver forbindelse). Udskift alle hardkodede værdier med Vault-opslag.

🚦 CI/CD & QA

Moderniser udviklingsflowet med fuld CI/CD og testautomatisering:

- **CI/CD-pipeline:** Opsæt GitHub Actions (eller Jenkins) workflows for *build/test/deploy*. Hver pull request skal udløse linting, unit- og integrationstests, samt statisk kodeanalyse (PHPStan/Psalm). Efter merge køres deploy til staging, evt. med kanary-release før produktion. Skabeloner og eksempler kan tages fra Blackbox-standard (jf. README's anbefaling om CI/CD). Rollback-mekanismer skal sikre hurtig genoprettelse ved fejl.
- **Automatiserede tests:** Udvikl PHPUnit-tests for alle forretningskritiske klasser og API-endpoints. End-to-end-tests (f.eks. med Behat eller Cypress) for brugerflows: login, agent-håndtering, mission-flows, tools-eksekvering. UI-tests sikrer, at dashboard og formularer fungerer. Pipeline skal mislykkes, hvis testcoverage falder.
- **Statisk kodeanalyse:** Inkluder værktøjer som PHP_CodeSniffer (PSR-12), PHPStan eller Psalm i CI. Ingen PR må merges, før alle check er grønne. Tilføj SAST (f.eks. SonarQube) for at opdage sikkerhedsbrister eller datavalideringsfejl.
- **QA-workflow:** Definer formelle procestrin: kodegennemgang (peer review), NEX-godkendelse (arkitekt) før merge til main, QA-agents validerer features mod krav. Brug GitHub Issues/projektboards til sprint-backlog og tracking. AI/Agent kan levere tjeklister og automatiserede audits (f.eks. overholdelse af kodestandard).
- **Sikkerhedstest:** Udfør regelmæssige penetrationstest og sårbarhedsscanning af alle nye releases. Skift til HTTPS (TLS) overalt, patch PHP og afhængigheder (brug Dependabot/Snyk). Log testresultater i rapporter.

🔐 Sikkerhed og Compliance

Robuste sikkerhedsforanstaltninger er altafgørende:

- **Autentificering:** Brug OAuth2/JWT for alle API'er. Overvej single sign-on, så BCP og AIG deler samme auth-server eller bruger fælles Vault/KeyVault. Støt 2FA og "ghost mode" (agenter kan få midlertidige stealth-sessioner).
- **Adgangskontrol:** Indfør streng rollebaseret politik: Admin vs Agent (jvf. README). Sikre at API'er og UI kun viser data til behørige roller. Log alle forsøg på uberettiget adgang.
- **Kryptering:** Brug TLS for data-in-transit, og krypter følsomme data i databasen (f.eks. brugerprofiler, logs) med server-side encryption. Pseudonymiser audit-logs for at efterleve privacy-by-design.
- **Vault Integration:** Implementér HashiCorp Vault (eller Azure Key Vault) til secrets-management. Alle applikationer henter deres credentials dynamisk fra Vault, og permissions styres via Vault-roller og politikker. Så undgås hardkodede nøgler i koden.

- **Audit og GDPR:** Log alle brugerhandlinger (opret agent, opdater mission, login mm.) i revisionsklare audit-trails. Tilbyd anonymiseringsmuligheder på datafelt-niveau. Sørg for at dataretention og sletningsprocedurer følger GDPR. Dokumentér compliance-krav i driftshåndbog.

🔗 **Integration med Blackbox-moduler**

AIG skal kunne samarbejde med eksisterende Blackbox-systemer:

- **Blackbox Control Panel (BCP):** Design en fælles autentificeringsmodel (f.eks. fælles OAuth2-server eller delt JWT-klient) så AIG og BCP kan udveksle gyldige tokens. Udvid AIG med nye endpoints under /api/integrations/bcp/ til indtagelse af BCP-hændelser (missioner, logs). BCP kan f.eks. kalde AIG's mission-API med JWT-token for at tildele missioner. Omvendt kan AIG kalde BCP's API (via REST eller webhook) for at trigge fysiske operationer. Planlæg en eventbus (MQTT eller Redis) eller WebSocket-kanal til push af live-data mellem systemerne.
- **ID-MATRIX:** Forvent at ID-Matrix håndterer agent-identifiering og session-audit. Udnyt deres API til verifikation af agentkrypteringsnøgler eller ID-certifikater. API-gateway i AIG kan forwarde autentificeringsforespørgsler til ID-Matrix. Log eventuelle login-forsøg via ID-Matrix for fælles rapportering.
- **Blackbox Prompts v1:** AIG's dashboard skal kunne udstille prompts/kontekst hentet fra BBP. Benyt et modul, der kalder BBP's REST API for prompt-data, og integrer svar via GreyEYE (AI Engine). Brug samme tokenbaserede auth. Design workflow, hvor AIG's front-end kan sende prompt-spørgsmål til BBP og vise AI-svar.
- **GreyEYE:** Udnyt GreyEYE som AI-mæglerservice: Brug WebSocket eller REST til at forespørge machine-learning-modeller (f.eks. OSINT-analyse). Installer en GreyEYE-klient på backend, så intelligence-data automatisk sendes til GreyEYE for bearbejdning. AIG skal også kunne modtage push-beskeder (via WS) med AI-genererede alerts eller mission-opdateringer.
- **General Integration:** For hvert modul identificér dataflader (JSON-skema) og konventioner, der matcher AIG's API-stil (f.eks. application/json; charset=utf-8, JWT-bearer-header). Udarbejd OpenAPI-specifikationer for nye integrations-endpoints, så eksterne teams (BCP, ID-Matrix-udviklere, mfl.) har klar dokumentation.

📚 **Dokumentation og versionsstyring**

Al dokumentation skal være fuldstændig, tilgængelig og versionsstyret:

- **SYSTEM_BLUEPRINT.md:** Udarbejd og vedligehold et udførligt blueprint-dokument (dette dokument). Det skal indeholde arkitekturdiagrammer (loginflow, modulkart) – fx genereret med Mermaid/PlantUML – samt opdateret beskrivelse af alle API'er, datamodeller og integrationspunkter. Merk dokumentets forside med titel SYSTEMBLUEPRINT & HANDLINGSPLAN – ALPHA Interface GUI, versionsnummer (v1.0), dato, ansvarlig (AlphaDEV/AI) og godkendelse (Agent NEX).
- **README.md:** Opdater hoved-README med detaljeret installationsvejledning (trinvist), afhængigheder, samt oversigt over alle moduler. Indsæt referencer til blueprint og andre dokumenter (CHANGELOG, onboarding). Gør det nemt for nyudviklere at komme i gang.
- **Operational manual:** Skriv en driftshåndbog for DevOps/Support, inkl. skridt-for-skridt instruktioner til installation, konfiguration (Vault, DB, TLS-certifikat), fejlsøgning og recovery. Inkluder typiske fejlkoder og løsninger.
- **CHANGELOG.md:** For hver hovedversion (major/minor) i projektet skal der føres en changelog med dato, forfatter og beskrivelse af ændringer. Følg semver-konventioner (se nedenfor). Dokumentér breaking changes tydeligt.
- **API-dokumentation:** Generér OpenAPI/Swagger-specifikationer for alle REST-endpoints (brug automatiske værktøjer eller manuelle YAML). Udgiv dem i /docs/ eller indlejret i GitHub via Swagger-UI.
- **Rapporter og bilag:** Saml alle statusrapporter, kvalitetstjek, kodeanmeldelser og referencer i /docs/reports/ med versionsnummer og dato i filnavnet. Husk at opsummere ansvarlig og signatur i hver rapport.

🗓️ **Roadmap og tidsplan**

Lav en detaljeret køreplan for implementering:

- **Fase 1 (0–4 uger):** Sæt op med basis-infrastruktur: Database, Vault, CI/CD (staging + prod). Påbegynd udvikling af Missions- og Intelligence-moduler.

- **Fase 2 (5–8 uger):** Udvid med Tools-modul, og implementér 2FA-login samt ghost mode. Udrul grundlæggende tests og code reviews.
- **Fase 3 (9–12 uger):** Udfør integrationstest med BCP og ID-Matrix (auth-model, API-kald). Byg WebSocket-eventbus (evt. via GreyEYE-broker). Kør penetrationstest og sikkerhedsrevision.
- **Fase 4 (13–16 uger):** Færdiggør dokumentation (blueprint, manualer, swagger) og onboard nye teammedlemmer via guider. Implementér canary-deploy og overhold semver ved udgivelse. Afsluttende QA-gennemgang og NEX-godkendelse.
- (Se evt. vedhæftet Gantt-diagram i bilag for detaljeret tidslinje.)

📋 Handlings- og opgaveliste

Nedenstående tabel opsummerer alle nødvendige opgaver. Hver kan oprettes som GitHub Issue med ansvarlig, deadline og status.

Opgave	Ansvarlig	Beskrivelse
Missions-modul	AlphaDEV	Implementér UI og API (opret/søg/tildel missioner). Tildel routes (GET/POST /api/missions).
Tools-modul	AlphaDEV	Udvikl UI for stealth-værktøjer og tilhørende API-endpoints for eksekvering og logging af værktøjskald.
Intelligence-modul	AlphaDEV	Implementér datafeeds/UI for OSINT-rapporter og kort. Inkluder kald til eksterne AI/OSINT-API'er.
Vault/Secrets-integration	AlphaDEV	Integrér HashiCorp/Azure Vault: Udskift alle hardkodede secrets (DB, API-nøgler) med Vault-opslag.
2FA/token-login & Ghost Mode	AlphaDEV	Tilføj multifaktor-login på /api/login, og implementer "ghost mode"-funktion for agenter.
Databaseskema og scripts	AlphaDEV	Dokumentér og script hele SQL-databasen (agenter, missioner, logs, mv.). Lav migrations og seed-data.
Logging/audit	AlphaDEV	Forbedr logning: central logging-klasse for alle hændelser (login, mission, osv.), krypteret audit-trail.
CI/CD-pipeline (GitHub Actions/Jenkins)	AlphaDEV	Opsæt automatiseret build/test/deploy til staging/prod. Inkluder rollback og kanary-release-strategi.
Automatiserede tests (unit/integration/UI)	AlphaDEV	Skriv PHPUnit-tests for API & business logic. Tilføj end-to-end UI-tests for login, missioner, logs.
Statisk kodeanalyse (PSR-12, SAST)	AlphaDEV	Inkluder PHP_CodeSniffer, PHPStan/Psalm og Snyk i CI. QA afvikling kræver grøn build før merge.
QA-plan & Review-workflow	AI/Agent	Definér review-proces og tjeklister. Dokumentér QA-agentens rolle, krav til NEX-godkendelse og compliance.
Sikkerhedstest & audit	AI/Agent/Dev	Planlæg løbende penetrationstest. Udarbejd audit-rapporter og security-check.
BCP-API-integration (design)	AI/Agent	Design fælles auth-model (OAuth2), nye endpoints (/api/integrations/bcp/...) og dataflow til BCP.
BCP-endpoints i AIG	AlphaDEV	Implementér BCP-webhooks: Mottag BCP-events (missioner, logs) via dedikerede API-ruter.
Auth-model mellem AIG & BCP	AlphaDEV/AI	Implementér delt JWT/OIDC-løsning eller Vault-baseret token-udstedelse (AI udarbejder design).
Dashboard-widgets for BCP-data	AI/Agent	Design UI-komponenter til at vise BCP-status og logs i AIG's dashboard.
Eventbus/WebSocket-plan	AI/Agent	Arkitekter realtidsintegration (eksempelvis Redis/MQTT). Beskriv flow for dataudveksling mellem systemer.
Forbered integration hooks (ID-Matrix, BBP, GreyEYE)	AI/Agent	Beskriv fælles SDK/API-mønstre for fremtidige integrationer. Forbered events eller classes til deling.
Udarbejd SYSTEM_BLUEPRINT.md	AI/Agent	Skriv og finalisér dette blueprint-dokument med diagrammer, beskrivelser, ansvar og procesflows.

Opdater README.md	AI/Agent	Udvid README med installationsvejledning, beskrivelser af moduler, og links til blueprint/CHANGELOG.
Driftshåndbog & Fejlsøgning	AI/Agent	Sammensæt driftmanual: guides til opsætning, backup, recovery, og typiske fejlsituationer.
CHANGELOG.md	AI/Agent	Versionér alle frigivelser; dokumentér ændringer, datoer og ansvarlig for hver version.
Swagger/OpenAPI-dokumentation	AI/Agent	Udfærdig API-spec for alle endpoints. Inkluder eksempler på forespørgsler/svar.
Kanary-deployment & rollback	AlphaDEV	Implementér kanary-tests (små produktionstests) og scripts til øjeblikkelig rollback.
SemVer-politik & tagging	AI/Agent	Definér versionsregler (MAJOR.MINOR.PATCH) og sørg for semantisk tagging af kode og dokumenter.
Backup-scripts	AlphaDEV	Opret automatiske backup-rutiner for DB, logs og konfig, gemt sikkert (krypteret) og versionsstyret.
Roller og ansvar	AI/Agent	Dokumentér alle roller (NEX, AlphaDEV, QA, Support) med deres ansvar. Opret approvals-workflow.
Onboarding-guide	AI/Agent	Sammensæt komplet onboarding for nye udviklere: kontaktpersoner, tech-stack, vigtigste funktioner.
Agile backlog & sprintplan	AlphaDEV	Brug GitHub Issues/Projects til sprintplanlægning. Afhold stand-ups, reviews og retros efter plan.
Løbende security reviews	AI/Agent	Integrér automatiske sikkerhedsgennemgange (Dependabot/Snyk). Auditér kodebasis regelmæssigt.
GDPR og privacy-by-design	AlphaDEV	Implementér dataminimering og logger med pseudonymisering. Sørg for høring af privacy-eksperter ved behov.

(Alle opgaver skal spores som issues/milepæle og løbende opdateres.)

👥 Roller og ansvarsfordeling

- **Agent NEX (arkitekt/godkender):** Overordnet ansvar for systemdesign og final approval. Sikrer compliance med Blackbox-standarder og godkender roadmap.
- **AIG AlphaDEV (udviklingsteam):** Implementerer alle moduler, CI/CD, sikkerhed og integrationer. Ansvarlig for kodestandard, test, deployment og daglig opdatering af blueprint.
- **QA-Agent:** Udfører uafhængige tests, kontrollerer kode og dokumentation mod krav. Ansvarlig for kvalitetssikring af nye releases (inkl. sikkerheds- og performance-test).
- **AI/Agent (support):** Udarbejder dokumenter, QA-planer og tjeklister. Hjælper med prototype-design, automatisering (pipelines, audits) og træningsmaterialer. Kan udarbejde analyser og løsningsforslag til svære problemstillinger.
- **Governance:** Alle kodeændringer skal gennemgå peer review og NEX-godkendelse. Ved merge til main udføres automatisk deployment til staging, med endelig release efter NEX-bekræftelse.

📎 Bilag og referencer

- **Opsætningsfiler:** /docs/SYSTEM_BLUEPRINT_AIG.md (dette dokument), /docs/SYSTEM_BLUEPRINT_AIG_v1.0.pdf.
- **Kode-/Konfig:** Se README i repo for krav og mappestruktur. Se /docs/CHANGELOG.md, /docs/OPERATIONAL_STATUS.md.
- **Diagrammer:** Flowdiagrammer for login, missions, dataflow og integrationsarkitektur (vedlagt eller genereret med fx Mermaid).
- **API-specifikation:** OpenAPI/Swagger JSON eller YAML for alle endpoints.
- **Rapporter:** Tidligere status- og kontrolrapporter i /docs/reports/.

Alle dokumenter og artefakter er versionsstyret i Git med semantisk tagging. Denne blueprint-version er udgivet som

v1.0 (2025-07-01) og godkendt af Agent NEX. Fremtidige ændringer dokumenteres som v1.1, v1.2 osv. Blueprintet skal være master-referencen for hele AIG-projektet og Blackbox-økosystemet.